

Trabalho de Sistemas Distribuídos

Prof. Alessandro Vivas Andrade

Objetivo

Implementar um **sistema distribuído para controle de frequência em sala de aula utilizando QR Code**, permitindo que professores gerem QR Codes para cada aula e que os alunos registrem presença utilizando seus dispositivos móveis.

O sistema deverá garantir que o registro de presença ocorra **somente dentro da universidade**, utilizando validação de rede institucional (IP/NAT), geolocalização da sala e autenticação do usuário.

Além disso, o sistema deverá ser projetado para **suportar milhares de acessos simultâneos**, utilizando técnicas modernas de **sistemas distribuídos**, incluindo **cache Redis, filas de eventos (RabbitMQ ou Kafka) e arquitetura baseada em microserviços**.

Requisitos

1. Django (última versão)
2. Banco de Dados PostgreSQL
3. Python
4. Django REST Framework
5. Redis (cache e controle de presença)
6. RabbitMQ ou Kafka (fila de eventos)
7. Biblioteca para geração de QR Code
8. Frontend em JavaScript
9. Grupos de até 04 alunos
10. Linux

Parte 1: Autenticação, QR Code e CRUD

Autenticação

Implementação de um sistema de autenticação utilizando **django-allauth**.

O sistema deve possuir:

- Login e senha para acesso ao sistema
- Controle de acesso baseado em permissões para diferentes tipos de usuários:
 - Administradores
 - Professores
 - Alunos
- Criptografia de senhas
- Políticas de senha seguras
- Registro de auditoria para rastrear as ações dos usuários
- Página principal do sistema
- Dashboard personalizado por usuário

Modelos Principais

Sugestão de estrutura de dados.

Aluno

- nome
- matrícula
- email
- curso

Professor

- nome
- email
- departamento

Disciplina

- nome
- código
- professor responsável
- semestre

Sala

- nome
- prédio

- latitude
- longitude
- raio permitido (em metros)

Aula

- disciplina
- sala
- data
- horário de início
- horário de fim
- token do QR Code

Presença

- aluno
- aula
- horário do registro
- ip registrado
- latitude
- longitude
- status

Geração de QR Code

O professor poderá:

- criar uma aula
- gerar um QR Code único para a aula
- imprimir o QR Code e levar para sala de aula

O QR Code deve conter:

- identificador da aula
- token de segurança

Exemplo de link:

<https://sistema.edu/presenca?id=123&token=abc123>

Os alunos irão escanear o QR Code utilizando seus dispositivos móveis.

Registro de Presença

O aluno deverá:

1. escanear o QR Code
2. realizar login no sistema
3. permitir acesso à localização do dispositivo

O sistema deve validar:

- autenticação do aluno
- se o aluno já registrou presença
- horário da aula
- localização do aluno
- rede da universidade

Somente após todas as verificações a presença poderá ser registrada.

Verificação de Rede da Universidade

O sistema deve verificar se o aluno está conectado à rede da universidade.

Isso será feito através da verificação do **IP de origem da requisição**.

O sistema deve permitir presença apenas se o IP estiver dentro do **range de IP institucional**.

Caso contrário, a presença deve ser negada.

Verificação de Geolocalização

O sistema deverá capturar a localização do aluno utilizando a **API de geolocalização do navegador**.

A localização capturada deverá ser comparada com a localização da sala.

A presença só será registrada se o aluno estiver dentro do **raio permitido da sala**.

Sugestão de raio:

50 metros.

CRUD

O sistema deve permitir operações CRUD para:

- alunos
- professores
- disciplinas
- salas
- aulas
- presenças

Funcionalidades obrigatórias:

Criação

Cadastro de dados e geração de QR Code.

Leitura

Busca por:

- aluno
- disciplina
- professor
- data da aula

Atualização

Permitir edição de:

- disciplinas
- aulas
- registros de presença (somente professor)

Exclusão

Permitir exclusão apenas para usuários autorizados.

Parte 2: REST API com Django REST Framework

O sistema deve disponibilizar uma **API REST**.

Endpoints públicos

- lista de disciplinas
- lista de professores
- lista de aulas

Formato de resposta:

JSON

Endpoints restritos

CRUD autenticado para:

- alunos
- disciplinas
- aulas
- presenças

Exemplos:

POST /api/aulas

POST /api/presencas

GET /api/presencas

Arquitetura de Microserviços

O sistema deverá ser dividido em **múltiplos serviços independentes**, por exemplo:

Serviço de Autenticação

- login
- controle de usuários

Serviço de Aulas

- gerenciamento de disciplinas
- geração de QR Code

Serviço de Presença

- registro de presença
- validação de IP
- validação de geolocalização

Serviço de Relatórios

- estatísticas
- dashboards

Cada serviço deverá possuir sua própria API.

Uso de Redis (Cache)

Redis deverá ser utilizado para:

- cache de sessões
- controle de presença já registrada
- validação rápida de tokens do QR Code
- redução de acesso ao banco de dados

Exemplo:

presenca:aula_id:aluno_id

Isso evita múltiplos registros simultâneos.

Fila de Eventos (RabbitMQ ou Kafka)

O sistema deverá utilizar uma fila de mensagens para processar eventos de forma assíncrona.

Eventos possíveis:

- registro de presença
- geração de relatórios

- auditoria de acesso

Fluxo exemplo:

Aluno registra presença → evento enviado para fila → serviço processa evento → grava no banco.

Isso permite escalabilidade e desacoplamento dos serviços.

Escalabilidade

O sistema deve ser projetado para suportar **milhares de alunos acessando simultaneamente**.

Estratégias recomendadas:

- cache Redis
- processamento assíncrono com filas
- serviços independentes
- APIs desacopladas

Dashboard

Criar um dashboard com gráficos contendo:

- presença por disciplina
- presença por aluno
- presença por aula
- média de frequência

Ferramentas sugeridas:

- Chart.js
- Plotly

Frontend

Criar interface web utilizando JavaScript.

A interface deve permitir:

- geração de QR Code
- registro de presença
- visualização de relatórios

O frontend deve consumir a API do sistema.

Histórico de Alterações

O sistema deve registrar:

- quem realizou alterações
- qual alteração foi realizada
- data e hora da alteração